

PROJETO 0

PREPARAÇÃO DO AMBIENTE DE DESENVOLVIMENTO E SIMULAÇÃO

Visual Studio Code + PlatformIO + Wokwi

Curso Técnico em Desenvolvimento de Sistemas
Fundamentos de Eletroeletrônica Aplicada

MISSÃO Preparar o computador, compilar o primeiro firmware e confirmar o LED interno do Arduino Uno em simulação.

Manual do aluno • Setembro de 2026

Apresentação

O Projeto 0 prepara o ambiente que será usado nas próximas práticas. Você instalará as ferramentas, criará um projeto para Arduino Uno, escreverá um programa curto, compilará o firmware e o executará no simulador Wokwi.

Ao final, você deverá conseguir repetir o ciclo de trabalho sem depender de uma placa física: editar o código, prever o resultado, compilar, executar e comparar o comportamento observado.

ROTA DO PROJETO VS Code → PlatformIO → código-fonte → compilação → firmware → Wokwi → observação

O que estará funcionando ao final

- Visual Studio Code aberto e pronto para extensões.
- PlatformIO IDE instalado e inicializado.
- Projeto Projeto_0 configurado para Arduino Uno e framework Arduino.
- Arquivos firmware.hex e firmware.elf gerados.
- Wokwi instalado e preparado para ativação e configuração com diagram.json e wokwi.toml.
- Após a ativação individual, a simulação estará configurada para executar o firmware e o LED interno deverá piscar conforme as temporizações.

Antes de começar

- Computador com Windows e acesso à internet.
- Permissão para instalar extensões no VS Code.
- Espaço livre para baixar ferramentas do Arduino Uno.
- Uma pasta de trabalho que você consiga localizar depois.
- Conta Wokwi ou possibilidade de criar uma, pois a extensão solicita uma licença.

IMPORTANTE Nenhuma placa física é necessária nesta atividade. Toda a verificação acontece no computador e no simulador.

SEGURANÇA Instale somente as extensões identificadas neste manual. Confira o nome e o publicador antes de clicar em Install.

1 Visual Studio Code

O Visual Studio Code, ou VS Code, é o editor no qual vamos organizar os arquivos, escrever o programa e acionar as ferramentas de compilação e simulação.

1

Instale o VS Code, se necessário

Acesse code.visualstudio.com, escolha Download for Windows, execute o instalador e conclua as

2

3

telas apresentadas. Se o VS Code já abre normalmente, avance.

Abra o editor

No menu Iniciar, procure por Visual Studio Code e abra o programa. Aguarde até a janela principal aparecer.

Localize Extensions

Na barra vertical à esquerda, clique em Extensions. Atalho: Ctrl+Shift+X. Essa área instala recursos adicionais.

RESULTADO ESPERADO A tela Extensions apresenta uma caixa de pesquisa e uma lista de extensões.

2 PlatformIO IDE

IDE significa Ambiente Integrado de Desenvolvimento, do inglês Integrated Development Environment. O PlatformIO IDE acrescenta ao VS Code as ferramentas necessárias para criar e compilar projetos embarcados.

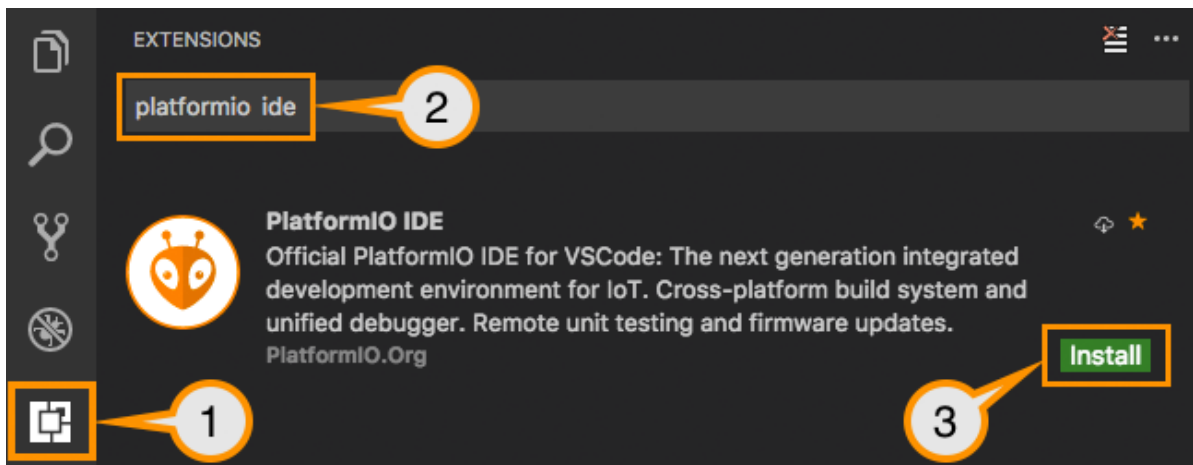


Figura 1 — Captura oficial da documentação atual do PlatformIO: Extensions, pesquisa e botão Install.

1

2

Pesquise a extensão

Na caixa de pesquisa de Extensions, digite PlatformIO IDE.

Confirme a extensão correta

Escolha PlatformIO IDE, publicada por PlatformIO.

3

Não instale uma extensão apenas porque o nome é parecido.

Instale

Clique em Install. Se o VS Code pedir confiança no publicador, confira novamente PlatformIO e confirme. Aguarde a conclusão.

SE NÃO FUNCIONOU Verifique a internet, a política de instalação do computador e se o VS Code precisa ser reiniciado.

Primeira inicialização do PlatformIO

A primeira abertura pode demorar porque o PlatformIO prepara componentes internos. Não feche o VS Code enquanto o indicador de atividade estiver trabalhando.

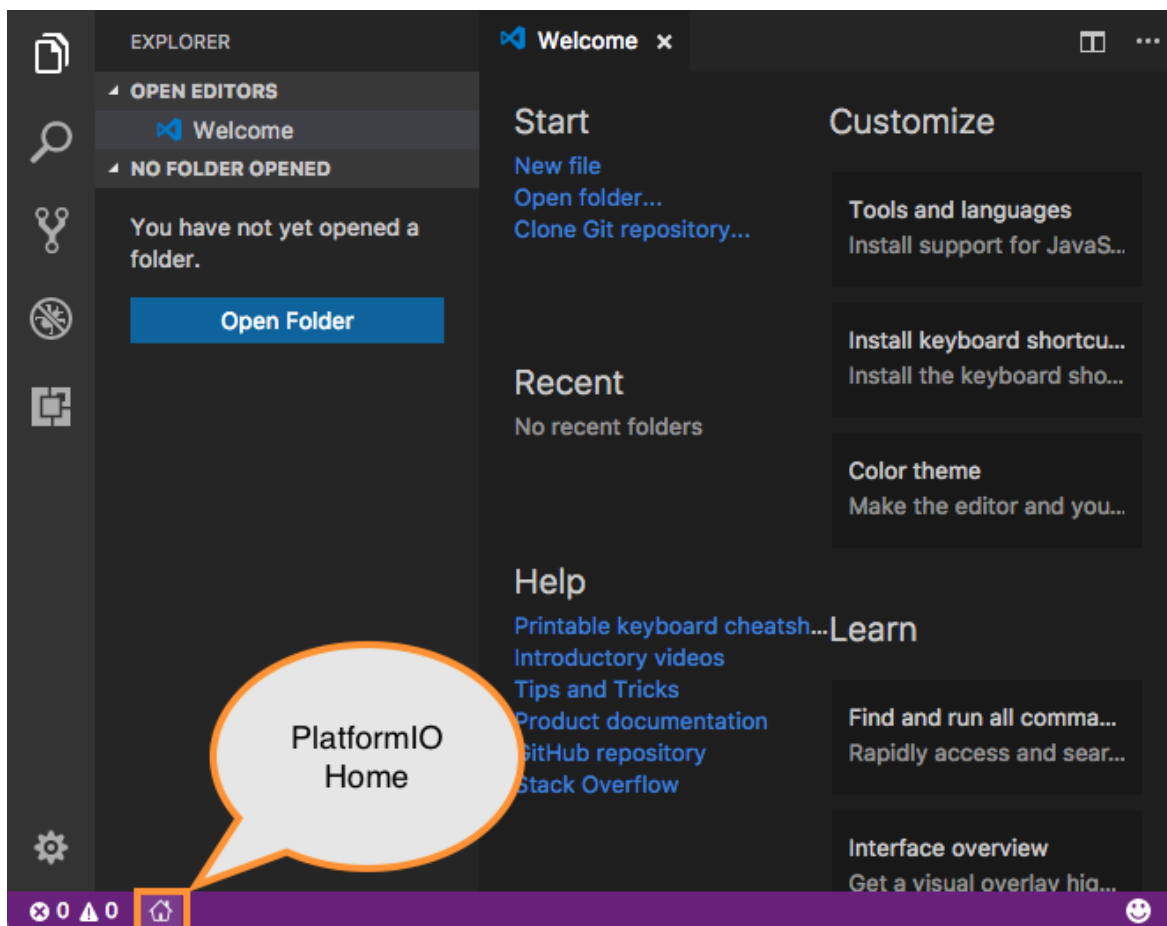


Figura 2 — PlatformIO Home conforme a documentação oficial atual.

1

2

Abra o PlatformIO Home

Clique no ícone da cabeça de alienígena na barra lateral e procure Quick Access → PIO Home → Open. Em algumas disposições, o ícone Home aparece também na barra inferior.

Aguarde a tela inicial

Espere até aparecer o PlatformIO Home. Na primeira execução, esse processo pode levar alguns minutos.

RESULTADO ESPERADO A página Home mostra atalhos como New Project, Open Project e exemplos.

3 Criando o Projeto 0

1

Inicie um novo projeto

No PlatformIO Home, clique em New Project.

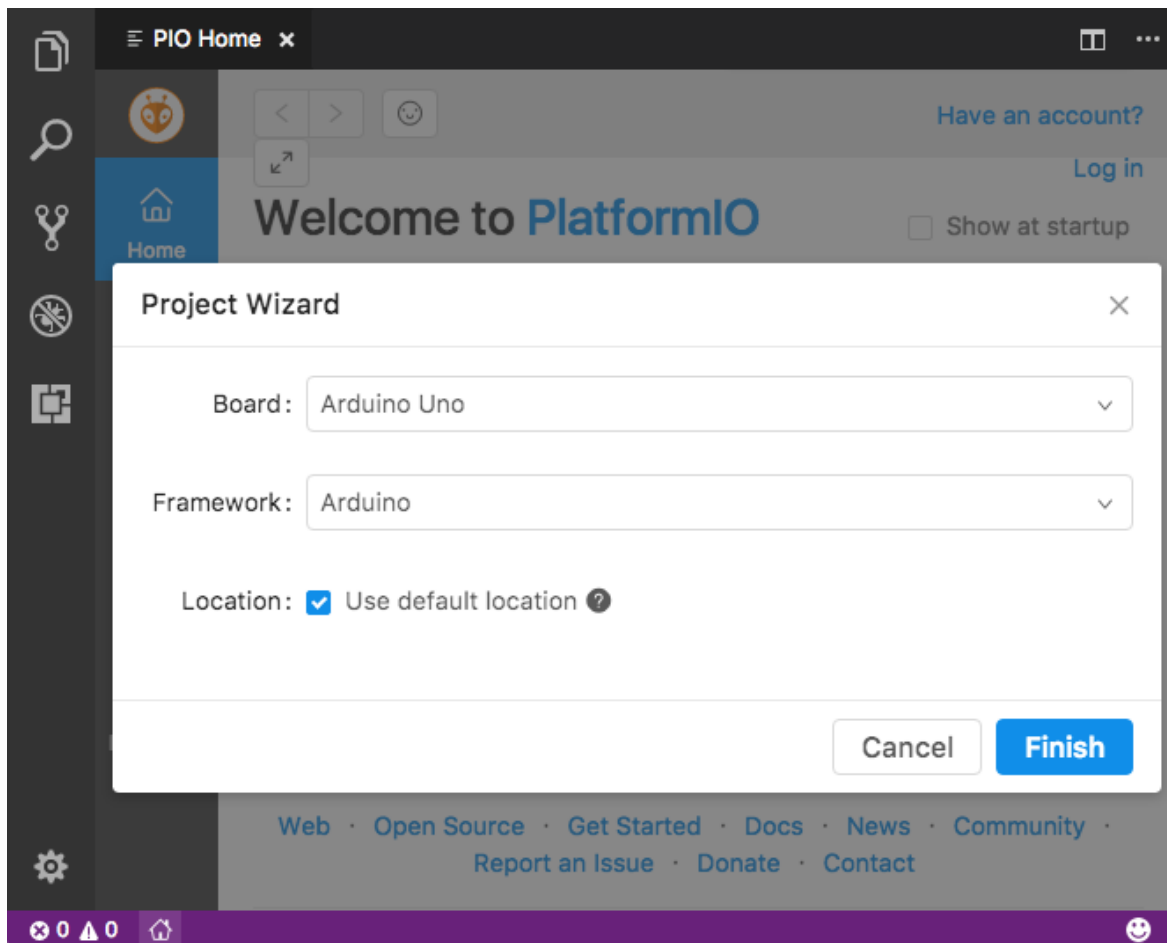


Figura 3 — Janela Project Wizard da documentação oficial do PlatformIO.

2

Preencha o projeto

Name: Projeto_0. Board: pesquise e selecione Arduino Uno. Framework: selecione Arduino.

3

Escolha a localização

Para facilitar esta atividade, desmarque Use default location e escolha uma pasta conhecida, ou mantenha a pasta padrão e anote onde o projeto foi salvo.

4

Crie o projeto

Clique em Finish. Aguarde o PlatformIO baixar ou localizar os pacotes do Arduino Uno.

RESULTADO ESPERADO O Explorer exibe a pasta Projeto_0 e seus arquivos.

DICA Se o assistente parecer parado, observe a barra inferior e o painel de notificações. A primeira criação costuma ser mais lenta.

4 Conhecendo a estrutura

No Explorer do VS Code, a estrutura básica será semelhante a esta:

```
Projeto_0/  
├── include/  
├── lib/  
├── src/  
│   └── main.cpp  
├── test/  
└── platformio.ini
```

Item	Para que serve agora
src	Guarda o programa principal.
main.cpp	Arquivo em C++ que contém setup() e loop().
platformio.ini	Define a placa, a plataforma e o framework.
include, lib e test	Existem para usos futuros; não precisamos aprofundá-los agora.

ATENÇÃO main.cpp deve permanecer dentro de src. Se estiver na raiz do projeto, o PlatformIO pode não compilar o código esperado.

5 O arquivo platformio.ini

Clique em platformio.ini no Explorer. Para este projeto, confirme o conteúdo essencial:

```
[env:uno]  
platform = atmelavr  
board = uno  
framework = arduino
```

[env:uno]

Cria um ambiente de compilação chamado uno.

platform = atmelavr

Seleciona a plataforma para microcontroladores AVR, família usada pelo Arduino Uno.

board = uno

Seleciona a placa Arduino Uno.

framework = arduino

Ativa as funções conhecidas do ambiente Arduino.

VERIFIQUE As palavras devem estar escritas exatamente como no bloco. Salve com Ctrl+S.

6 Primeiro programa

Abra src → main.cpp, selecione o conteúdo existente e substitua pelo programa abaixo. LED significa Diodo Emissor de Luz, do inglês Light Emitting Diode.

```
#include <Arduino.h>

void setup() {
  pinMode(LED_BUILTIN, OUTPUT);
}

void loop() {
  digitalWrite(LED_BUILTIN, HIGH);
  delay(1000);

  digitalWrite(LED_BUILTIN, LOW);
  delay(1000);
}
```

AÇÃO DO ALUNO Digite ou cole o código em main.cpp e pressione Ctrl+S.

Entendendo somente o necessário

#include <Arduino.h> — Disponibiliza as funções e constantes do framework Arduino.

setup() — Executa uma vez quando a placa inicia.

loop() — Repete continuamente enquanto o programa está em execução.

pinMode() — Define como um pino será utilizado.

LED_BUILTIN — Nome do LED interno da placa, ligado ao pino 13 no Arduino Uno.

OUTPUT — Configura o pino como saída.

digitalWrite() — Define o estado elétrico da saída.

HIGH e LOW — HIGH liga a saída; LOW desliga a saída.

delay() — Mantém o programa aguardando pelo tempo informado.

milissegundo — Mil milissegundos equivalem a um segundo.

NÃO PRECISA DECORAR O objetivo é reconhecer o papel de cada comando e acompanhar o fluxo. Aprofundaremos programação quando ela for necessária.

7 Compilando

Compilar significa transformar o código-fonte escrito por você em arquivos que o microcontrolador e o simulador podem executar.

FLUXO CÓDIGO-FONTE → COMPILAÇÃO → FIRMWARE → EXECUÇÃO



Figura 4 — Barra do PlatformIO. O ícone de marca de verificação executa Build.

1

Salve o código

Pressione Ctrl+S em main.cpp.

2

Execute Build

Clique no ícone de marca de verificação da barra inferior. Alternativas: Ctrl+Alt+B ou Command Palette → PlatformIO: Build.

3

Leia o terminal

Acompanhe o painel Terminal. O processo termina com SUCCESS quando a compilação é concluída.

RESULTADO ESPERADO A última linha contém [SUCCESS] e não há linha final de erro.

Reconhecendo um build bem-sucedido

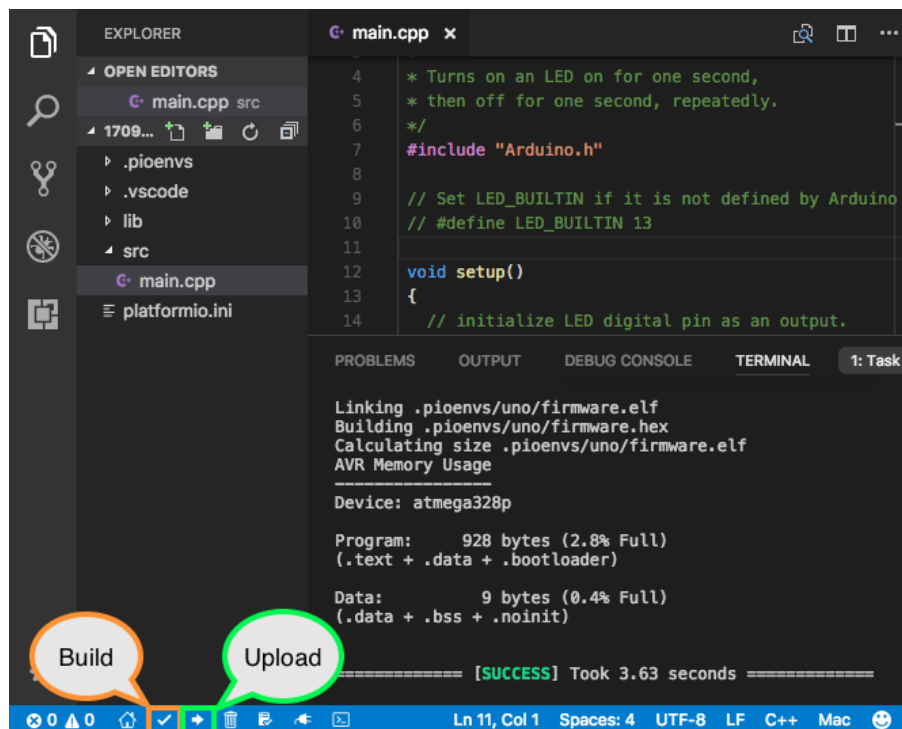


Figura 5 — Captura oficial do PlatformIO mostrando a área do projeto e a conclusão do Build.

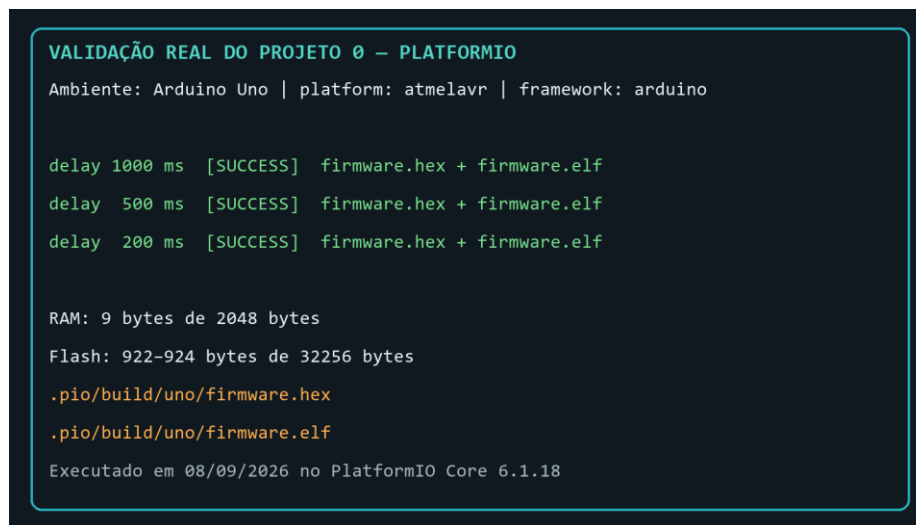


Figura 6 — Registro da validação técnica executada para este manual.

Depois do Build, o PlatformIO cria uma pasta oculta chamada .pio. No ambiente uno, os arquivos usados pelo Wokwi ficam em:

```
.pio/build/uno/firmware.hex
```

```
.pio/build/uno/firmware.elf
```

ATENÇÃO Não edite os arquivos de firmware. Eles são recriados a cada compilação.

8 Instalando e preparando o Wokwi

O Wokwi é um simulador de eletrônica e sistemas embarcados. A extensão Wokwi Simulator executa, dentro do VS Code, o firmware compilado pelo PlatformIO.

1

Abra Extensions

Clique em Extensions ou pressione Ctrl+Shift+X.

2

Pesquise Wokwi

Digite Wokwi Simulator. Confirme que o publicador é Wokwi e que o identificador é wokwi.wokwi-vscode.

3

Instale a extensão

Clique em Install e aguarde.

4

Solicite a licença

Pressione F1 e execute Wokwi: Request a New License. Autorize a abertura do site, entre em sua conta Wokwi e conclua o envio da licença ao VS Code.

5

Confirme a ativação

Volte ao VS Code e procure a mensagem License activated. A licença é individual e pode exigir autenticação pela internet.

SE A ESCOLA BLOQUEAR O LOGIN Registre o computador e avise o professor. Não compartilhe senha ou licença de outra pessoa.

9 Criando diagram.json

Na raiz de Projeto_0, no mesmo nível de platformio.ini, crie diagram.json. Clique com o botão direito na pasta Projeto_0 → New File → digite diagram.json → Enter.

```
{  
  "version": 1,  
  "author": "Projeto 0",  
}
```

```
"editor": "wokwi",
"parts": [
  {
    "type": "wokwi-arduino-uno",
    "id": "uno",
    "top": 0,
    "left": 0,
    "attrs": {}
  }
],
"connections": [],
"dependencies": {}
}
```

parts descreve os elementos da bancada virtual. Aqui existe somente um Arduino Uno. connections fica vazio porque o LED utilizado já está integrado à placa e não há componentes externos.

VERIFIQUE O arquivo se chama diagram.json, sem .txt no final, e está na raiz de Projeto_0.

10 Criando wokwi.toml

Ainda na raiz de Projeto_0, crie wokwi.toml. Esse arquivo informa ao Wokwi qual firmware deverá ser executado.

```
[wokwi]
version = 1
firmware = '.pio/build/uno/firmware.hex'
elf = '.pio/build/uno/firmware.elf'
```

version = 1 — Versão atual do formato de configuração.

firmware — Aponta para o arquivo Intel HEX gerado no Build.

elf — Aponta para o arquivo ELF, que contém informações úteis para a simulação.

ATENÇÃO Os caminhos são relativos à raiz do projeto e dependem do nome do ambiente [env:uno]. Se o ambiente tiver outro nome, ajuste a pasta depois de .pio/build/.

11 Primeira simulação

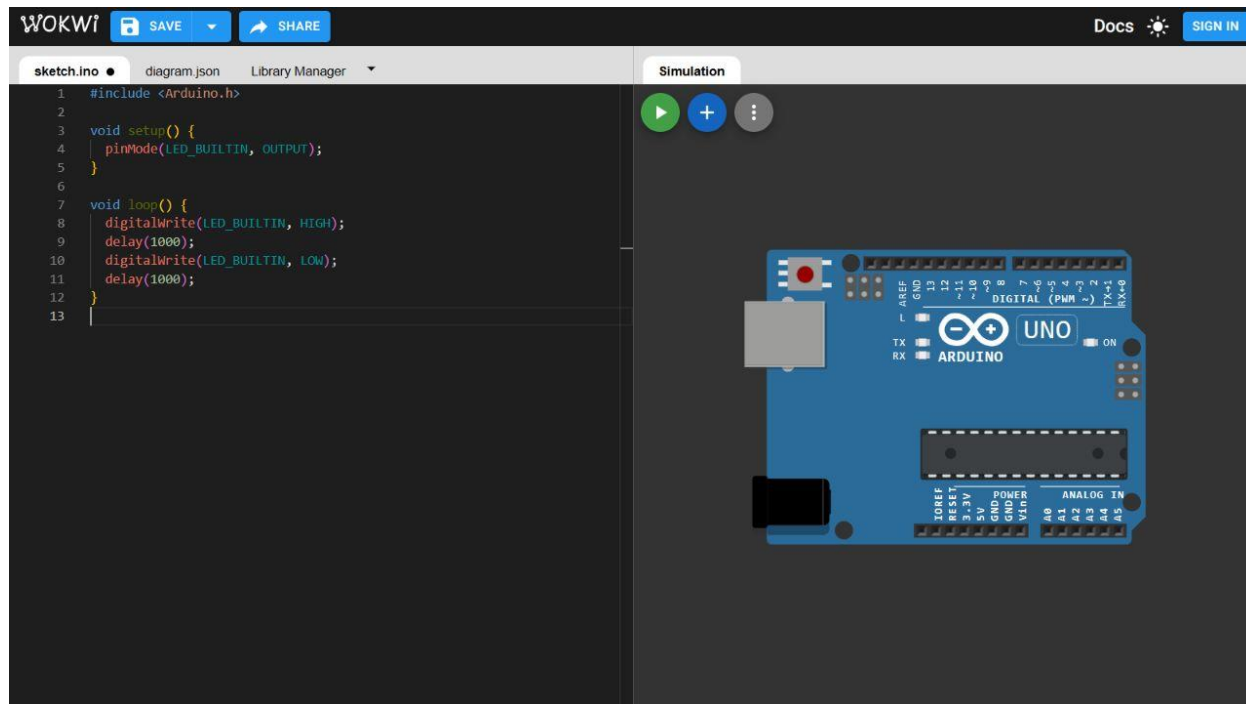


Figura 7 — Captura real da interface atual do Wokwi com o Arduino Uno e o LED interno L.

1

2

3

Compile novamente

Execute PlatformIO: Build e confirme SUCCESS. Isso garante que o firmware contém a última versão do código.

Inicie o simulador

Pressione F1, digite Wokwi: Start Simulator e pressione Enter. Também é possível abrir diagram.json e usar o botão verde de execução quando o editor visual estiver disponível.

Localize o LED interno

Na placa Arduino Uno, localize a indicação L. Observe por vários segundos.

RESULTADO ESPERADO LED ligado por aproximadamente 1 segundo → LED desligado por aproximadamente 1 segundo → repetição contínua.

12 Modificar prever executar e observar

MÉTODO MODIFIQUE → PREVEJA → COMPILE → EXECUTE → OBSERVE → COMPARE → CONCLUA

Teste com 500 ms

1

Modifique

Em main.cpp, troque as duas ocorrências de delay(1000); por delay(500);

ANTES DE EXECUTAR O que você prevê que acontecerá com o ritmo do LED?

2

Compile

Salve e execute Build. Aguarde SUCCESS.

3

Execute e observe

Reinicie o Wokwi. Compare o novo ritmo com o teste de 1000 ms.

RESULTADO ESPERADO O LED alterna duas vezes mais rápido: cerca de meio segundo ligado e meio segundo desligado.

Teste com 200 ms

1

Modifique

Troque as duas ocorrências de delay(500); por delay(200);

ANTES DE EXECUTAR O LED parecerá lento, igual ou mais rápido? Registre sua previsão antes do Build.

2

Compile e execute

Salve, execute Build, confirme SUCCESS e reinicie o Wokwi.

3

Compare

Observe o ritmo de 200 ms e compare com 500 ms e 1000 ms.

RESULTADO ESPERADO O LED alterna rapidamente: cerca de 0,2 segundo ligado e 0,2 segundo desligado.

CONCLUSÃO Reduzir o valor de delay reduz o tempo de espera e aumenta a frequência aparente das mudanças.

13 Diagnóstico rápido

SINTOMA	CAUSA PROVÁVEL	O QUE VERIFICAR
PlatformIO não apareceu	Extensão ausente, desativada ou inicializando	Extensions → PlatformIO IDE; confira Installed e aguarde a primeira inicialização.
New Project demora	Download inicial de pacotes	Internet, notificações do VS Code e atividade na barra inferior.
Build termina com erro	Código não salvo ou erro de digitação	Salve main.cpp; leia a primeira linha de erro; compare chaves, parênteses e ponto e vírgula.
main.cpp não é compilado	Arquivo fora de src	Confirme Projeto_0/src/main.cpp.
firmware não existe	Build ainda não concluiu com SUCCESS	Execute Build e verifique .pio/build/uno/.
Wokwi não encontra firmware	Caminho incorreto no wokwi.toml	Compare o nome do ambiente e os caminhos firmware e elf.
Simulador não inicia	Extensão ausente, licença não ativada ou conexão indisponível	Extensions → Wokwi; F1 → View License Information; confira a internet.
Mudança não apareceu	Firmware antigo ainda está em execução	Salve, compile novamente e reinicie a simulação.
diagram.json abre como texto	Editor visual indisponível no plano ou escolhido manualmente	O arquivo pode ser editado como texto; use F1 → Wokwi: Start Simulator.

Como ler um erro sem adivinhar

- Procure a primeira linha que contém error ou FAILED.
- Identifique o nome do arquivo e o número da linha, quando forem mostrados.
- Abra esse arquivo e compare a linha com o exemplo do manual.
- Corrija uma causa por vez.
- Salve e execute Build novamente.

NÃO FAÇA Não apague a pasta inteira nem altere vários arquivos ao mesmo tempo sem saber qual problema está corrigindo.

PEÇA AJUDA COM EVIDÊNCIA Informe ao professor o sintoma, o passo em que ocorreu e as últimas linhas do terminal.

14 Checklist final

- ☐ VS Code funcionando
- ☐ PlatformIO instalado
- ☐ Projeto_0 criado
- ☐ Arduino Uno selecionado
- ☐ Framework Arduino configurado
- ☐ main.cpp localizado
- ☐ Código inserido
- ☐ Build executado
- ☐ Compilação concluída
- ☐ Wokwi instalado e configurado
- ☐ diagram.json criado
- ☐ wokwi.toml criado
- ☐ Simulação iniciada
- ☐ LED interno piscando
- ☐ delay alterado para 500 ms
- ☐ Projeto recompilado
- ☐ Mudança observada
- ☐ delay alterado para 200 ms
- ☐ Resultado comparado com a previsão

15 Registro do aluno

Nome:

Data:

Computador:

Minha previsão com 500 ms:

O que observei:

Minha previsão com 200 ms:

O que observei:

O resultado foi igual ao previsto? () Sim () Não

Se não, o que aconteceu?

Qual foi a principal dificuldade durante a preparação do ambiente?

Como ela foi resolvida?

Resumo dos arquivos finais

```
Projeto_0/  
├── src/  
│   └── main.cpp  
├── diagram.json  
├── platformio.ini  
├── wokwi.toml  
└── .pio/build/uno/  
    ├── firmware.hex  
    └── firmware.elf
```

VOCÊ CONCLUIU O PROJETO 0 Seu ambiente está pronto quando todos os itens do checklist puderem ser marcados e o LED responder às três temporizações.

Referências

- **Visual Studio Code.** Extension Marketplace. <https://code.visualstudio.com/docs/configure/extensions/extension-marketplace>. Consulta em 8 set. 2026.
- **Visual Studio Code.** Installing Visual Studio Code on Windows. <https://code.visualstudio.com/docs/setup/windows>. Consulta em 8 set. 2026.
- **PlatformIO.** PlatformIO IDE for VSCode. <https://docs.platformio.org/en/latest/integration/ide/vscode.html>. Documentação latest consultada em 8 set. 2026.
- **PlatformIO.** Arduino Uno. <https://docs.platformio.org/en/latest/boards/atmelavr/uno.html>. Consulta em 8 set. 2026.
- **Wokwi.** Getting Started with Wokwi for VS Code. <https://docs.wokwi.com/vscode/getting-started>. Consulta em 8 set. 2026.
- **Wokwi.** Configuring Your Project wokwi.toml. <https://docs.wokwi.com/vscode/project-config>. Consulta em 8 set. 2026.
- **Wokwi.** diagram.json File Format. <https://docs.wokwi.com/diagram-format>. Consulta em 8 set. 2026.

- **Arduino.** Built-in Example Blink. <https://docs.arduino.cc/built-in-examples/basics/Blink/>. Consulta em 8 set. 2026.